

# **Credit Card Fraud Detection Using Machine Learning Techniques**

**Jasmina Abdullaeva**  
**Gianella Robles**

**Fordham University**  
**CISC 5790 Data Mining**  
**Dr. Li**

**April 2026**

## Abstract

---

Credit card fraud is a major problem for the world economy that results in a \$33-billion loss globally each year. We provide a cost-sensitive machine learning framework in this paper which aims at effectively detecting fraudulent credit card transactions, and minimizes loss as well as false positive rate for real customers. Using the real dataset of 1,852,394 transactions with high class imbalance (0.52% fraud), we test the performance of three different classification models, including Logistic Regression, Random Forest and XGBoost, by integrating them with SMOTE and random undersampling algorithms.

The specific focus is on the asymmetry of misclassification costs which is inherent to this problem, to optimize the classification thresholds so as to consider the actual business impact rather than pure accuracy. By conducting exploratory data analysis, we identify the significant indicators which can help identify fraudulent activities, such as category of transactions, time of day, and transaction amount. Performance measures we chose for the evaluation of classification results, such as Precision, Recall, F1-Score and AUC, are particularly sensitive to severely imbalanced data. According to the experiments, different models were tested and evaluated. The results show how model choice and data imbalance affect fraud detection performance.

## 1. Introduction

---

With the growth of digital payment systems, credit card fraud has become an increasing concern for both consumers and financial institutions. In recent years, global card fraud losses have reached over \$33 billion globally, and this is expected to grow even larger as card-not-present (CNP) transactions continue to expand in the age of e-commerce, as reported in the Nilson Report. Fraud transactions are rare compared to authorized ones, which makes detecting them challenging.

The foundation of any major card network or issuing bank is an automated fraud detection system. These systems must process decisions for millions of transactions daily within milliseconds, flagging them for review or denying their approval prior to settlement. This task boils down to an imbalanced binary classification problem: transactions that are fraudulent are exceedingly rare (less than 1% of total transactions), yet the implications of failing to catch a fraudulent transaction (a false negative) are enormous. Additionally, the chances of identifying a transaction as fraudulent can be another issue faced among experiments.

Standard machine learning techniques that aim for a higher overall accuracy will not work for this problem. A "dumb" classifier that labels all transactions as legitimate will obtain over 99% accuracy with 0% fraud detections. To tackle fraud, the focus of any system must shift from accuracy to managing class imbalance, asymmetric misclassification costs, and appropriate evaluation metrics that align with business objectives.

In this project, we attempt to tackle this problem with a cost-sensitive machine learning approach. We use a publicly available simulated credit card transaction dataset with 1,852,394 transactions in both training and test sets, with approximately 0.52% of which are fraudulent. Our objective is threefold: (1) to build models that achieve detection rates substantially higher than a simple baseline with a false positive rate below 1%; (2) to compare various resampling techniques that address class imbalance; and (3) to discover which features are most predictive of fraud.

## **2. Background**

---

Credit card fraud consists of various fraudulent behaviors in which an unknown person exploits card data or card account information for monetary benefit. In today's businesses, card-not-present (CNP) fraud, counterfeit card fraud, lost and stolen card fraud, and application fraud (where an account is created with fake information) are the most common types.

Fraud detection systems must make decisions very quickly, often within milliseconds. This means that models need to be efficient and able to produce predictions in real time. While some machine learning models may take longer to train. They are still useful because they can generate fast predictions once trained.

The problem of class imbalance is inherent to fraud detection. For most real-world situations, fraud rate lies between 0.1%-2% of total transactions. Standard machine learning algorithms trained on imbalanced data are biased to predict majority class cases, thus leading to an accuracy metric that doesn't truly represent a model's performance by always classifying every data point as legitimate. One needs to tackle this by either altering training data (resampling), or modifying learning algorithms (cost-sensitive learning), or both.

In fraud detection, approving a false negative fraudulent transaction costs the amount of the transaction plus chargeback fee and investigations expense. Declining a legitimate transaction loses customer satisfaction and potential customer, sometimes the merchant relationship. By assigning various costs to the wrong classifications, cost-sensitive learning allows model optimization that reflects true business compromise rather than an abstract accuracy goal.

## **3. Related Work**

---

There is extensive research on using machine learning techniques for fraud detection, with Dal Pozzolo et al. (2015) laying much of the initial groundwork. Dal Pozzolo et al. (2015) show that gradient boosting methods substantially outperform logistic regression on actual credit card fraud datasets when combined with a balanced sampling strategy. In particular, their work emphasizes the importance of calibrated probability estimates to be used for optimal threshold selection, a key aspect of our approach.

Chawla et al. (2002) present the Synthetic Minority Over-sampling Technique (SMOTE), which is one of the most common and widely adopted solutions to the class imbalance problem. SMOTE synthetically generates samples of the minority class by taking a new point along the line segment between two existing minority points in the feature space. This means the decision boundary for the fraud class is “pushed” out, as the number of fraud-like points increases in the proximity of existing ones. Later SMOTE variants (Borderline-SMOTE, ADASYN) adapt the sampling density such that more effort is placed on regions close to the class boundary, making it more effective to train on "hard" examples.

Japkowicz and Stephen (2002) perform a broad and comparative analysis on over- and undersampling strategies, concluding that optimal strategy choice heavily depends on the specific dataset and factors such as the degree of imbalance and the complexity of the decision boundary. Their results suggest that moderate oversampling of the minority class, combined with some undersampling of the majority class, yields the best results compared to individual over- or undersampling approaches.

More recently, Bhattacharyya et al. (2011) analyzed support vector machines and random forests on credit card fraud detection, and reported that random forests can achieve good results on these datasets with good performance and interpretability due to built-in feature importance. Since its proposal in 2016, Chen and Guestrin (2016) developed XGBoost, which has proven to be the state-of-the-art technique for tabular financial data Kaggle competitions, largely due to its speed, regularization techniques and handling of missing data.

#### **4. Exploratory Data Analysis**

---

Our research was based on the Kaggle dataset called “Credit Card Transaction Fraud Detection Dataset” containing both legitimate and fraudulent data from January 1st, 2019 to December 31st, 2020. The data covers 1000 customers with multiple transactions from different merchants. Various columns were included in the dataset for each transaction. Important features that will be investigated in this report include age, gender, job, transaction amount (amt), category (category\_clean), fraud label (is\_fraud), and so on.

Upon looking over the dataset, two csv files were presented but the main dataset was the fraudTrain. The columns were looked over and then dropped the unnecessary ones such as name, street, transaction id number (trans\_num), and credit card number (cc\_num). The remaining columns were examined to see what type of data object for decimals, integers, and objects (text category). Fraud detection was reviewed for a unique value count: 1 - 1,289,169, 0 - 7506. For this reason, columns were examined for any null or empty entries that needed to be dropped. The output gave a summary of each column and there was no need for dropping entries. In *Figure 1*, the graphs demonstrate how imbalanced the data is.

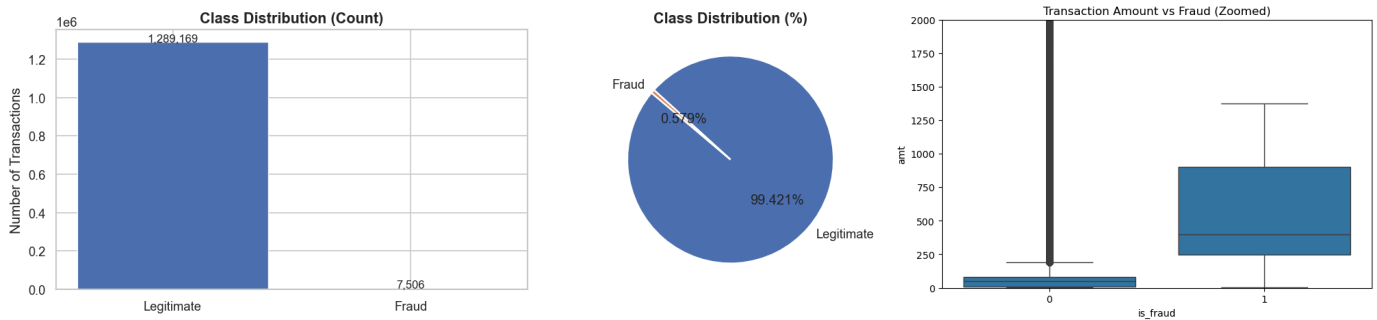


Figure 1

Looking through the data, the goal was to investigate any patterns that can impact the numbers for future models. The first column to investigate was gender, since it is only going to be Male vs Female. The count of females is higher than males in the raw data by 55% but the goal was to determine which gender is more likely to face fraud. Therefore, the code was grouped by the necessary column, `gender_rate = train_df.groupby('gender')['is_fraud'].mean()`. By using the `['is_fraud'].mean()`, the average of each detection (`is_fraud`) is calculated for each gender. This provides a normalized comparison, showing the proportion of fraudulent transactions within each group rather than total counts. As demonstrated on Figure 2, males show a slightly higher percentage of fraud compared to females by 0.10%. Since fraud is rare, the low percentage aligns, but gender isn't a strong indicator of it.



Figure 2

The age distribution shows the overall distribution of customer ages in the dataset, the majority is between individuals aged 30 to 60. However, fraud rate analysis reveals that older age groups, particularly those aged 65 and above have a higher proportion of fraudulent transactions. The boxplot compares the age distribution between fraud and not fraud transactions. Further supporting this pattern, the boxplot shows that fraudulent transactions tend to involve slightly older individuals. Regardless, there is a considerable overlap between the two groups, which indicates that age alone is not a strong predictor of fraud but may contribute when combined with other factors.

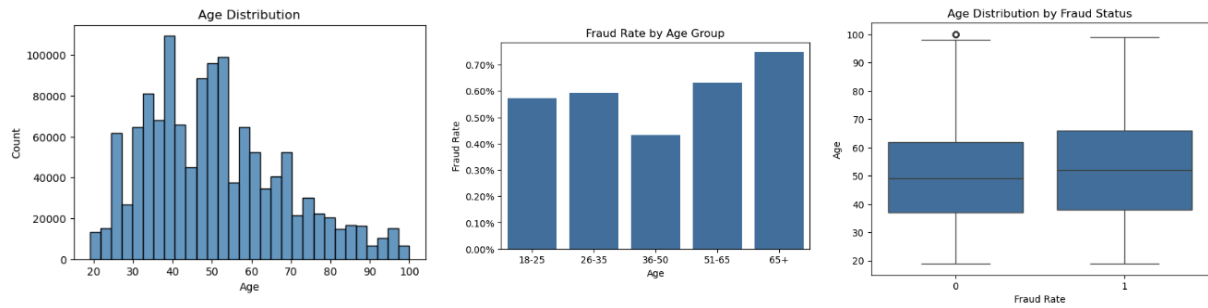


Figure 3

Fraud patterns were analyzed across time-based features, such as hour of day, day of week, and month. The hourly model shows that fraud rates are significantly higher during late-night and early-morning hours (UTC), particularly around midnight. This suggests that fraudulent activity is more likely to occur during off-peak times when people are not actively monitoring their devices or cards. Fraud rates across the days of the week indicate a gradual increase from Monday to Friday, with slightly lower rates during the weekend. Frauds tend to be more prevalent on weekdays than on weekends. The monthly trends show moderate variation, with slightly higher fraud rates in the early months of the year and again toward the fall season. Overall, these patterns suggest that time-based features contribute meaningful signals for fraud detection, although differences are not extreme and should be considered alongside other variables.

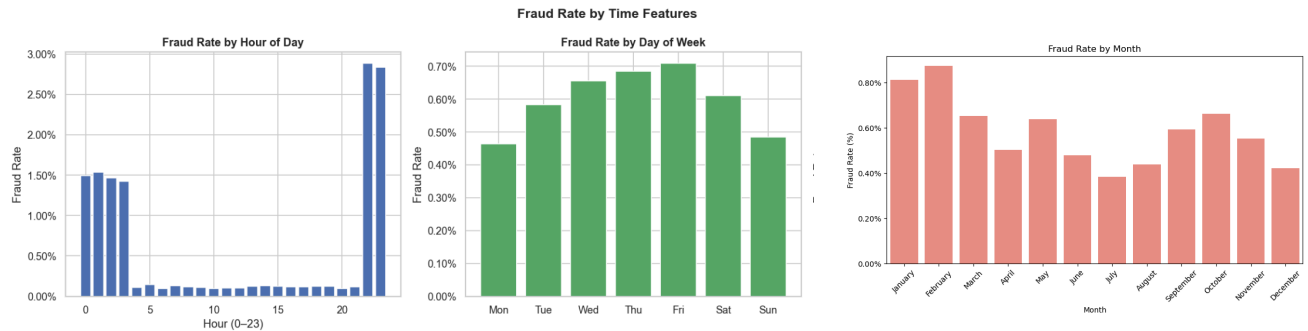
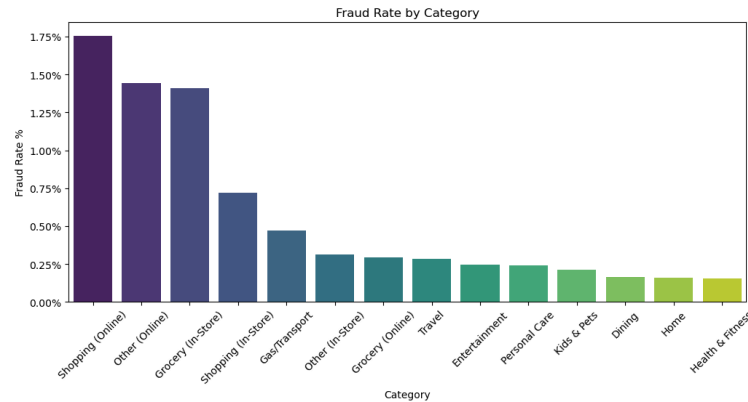


Figure 4

Additionally, fraud rate was analyzed across different transaction categories to identify which types of purchases are more susceptible to fraud activity. The results show that online transaction categories, such as shopping (online), and other (online), demonstrate the highest fraud rates compared to in-store transactions. Grocery-related transactions also show relatively elevated fraud rates, particularly for in-store purchases. In contrast, categories like dining, home, and health & fitness have noticeably lower fraud rates. Overall, these findings suggest that online transactions are more vulnerable to fraud, likely due to the lack of physical verification, while everyday in-person purchases tend to be less risky.



*Figure 5*

On Figure 6, this boxplot compares transaction amounts between fraudulent and non-fraudulent transactions. After zooming in, there seems to be a clear sign that fraudulent transactions tend to have higher amounts compared to normal transactions. Most non-fraud transactions are clustered at lower values, while fraud transactions show a wider and higher distribution. Although there is some overlap, this pattern suggests that transaction amount is an important feature, which aligns with our model results where it was the most influential variable.



*Figure 6*

After identifying all individual categories, it is time to have them compared to see how important each feature is. The correlation matrix shows the relationship between numeric features and fraud. Overall, most variables have very low correlation with the fraud label, indicating that no single feature strongly predicts fraudulent activity on its own. Transaction amount shows the highest positive correlation with fraud, though it is still relatively weak. Other features such as hour, age, and city population display minimal correlation with fraud. Additionally, some features are more correlated with each other such as day of the week and

weekend indicators, but these relationships do not strongly influence fraud detection. These results suggest that fraud is a complex problem driven by multiple factors, emphasizing the need for models like Random Forest and XGBoost that can capture interactions between features rather than relying on individual variables.

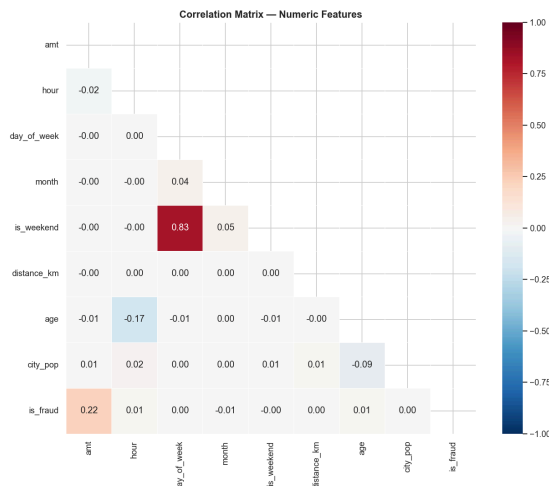


Figure 7

## 5. Experiments

### 5.1 Evaluation Metrics

Since class imbalance is severe, accuracy will not be our primary metric. We evaluate all models based on the metrics below which are considered as best practice in imbalanced classification for fraud detection:

Precision (Positive Predictive Value) tells us what fraction of transactions flagged as fraud were truly fraudulent. A low precision means a high fraction of normal transactions are blocked (high false positive rate), negatively impacting customer experience. We aim to keep the false positive rate under 1% thus demanding high precision.

Recall (Sensitivity) tells us what fraction of actual fraud transactions are detected. A low recall means much of the fraudulent transactions are able to slip through the detection system. We aim for a reduction in financial losses of 60% so we need at least 0.6 recall.

F1-Score is the harmonic mean of precision and recall and provides a single balanced metric of the performance. This metric is helpful for comparing models that adopt different imbalance handling strategies.

Precision-Recall AUC (PR-AUC) summarizes the model performance across all classification thresholds; the area under the PR-curve is insensitive to highly imbalanced data, unlike the ROC-AUC which is biased in such cases. This is our primary comparison metric.

Confusion Matrix inspection allows the analysis of raw counts of true positives, false positives, true negatives, and false negatives and calculation of the expected financial loss under certain cost ratios.

## **5.2 Models**

We implement and compare the performance of three classification models covering simple interpretable linear models to sophisticated state-of-the-art ensemble methods.

**Logistic Regression:** This linear discriminant model generates well calibrated probability scores and can be used with an optimization of threshold. We use L2 regularization ( $C=1.0$ ) and `class_weight='balanced'`. We scale all features to have zero mean and unit variance prior to fitting. Even though it is simple, this model is useful because of its interpretability. It serves as a lower bound to our expected performance.

**Random Forest:** This is a well-established ensemble model consisting of many decision trees trained using bootstrap samples and random subsets of features at each split. We train 200 trees with `maxdepth` set to 20 and `minsamplesleaf` set to 10. `Classweight='balanced_subsample'` is used so that classes have equal weight independently for each bootstrap sample. Random forests naturally control over-fitting by averaging predictions of individual trees and provide feature importance based on mean decrease in impurity.

**XGBoost (Extreme Gradient Boosting):** This regularized gradient boosting framework has demonstrated excellent performance in tabular financial fraud detection challenges. We use 300 estimators, a learning rate of 0.1 and max depth of 6. `Classweight` is used via `scaleposweight` which equals the ratio of majority to minority class. `Subsampling (0.8)` and `colsample_bytree (0.8)` parameters are added for regularization. We use early stopping based on validation PR-AUC to prevent over-fitting.

## **5.3 Experimental Setup**

The dataset was separated into training and testing sets to evaluate model performance while preventing data leakage. To address class imbalance, we applied class weighting, undersampling, and SMOTE to the training data only, while keeping the test set unchanged.

We implemented Logistic Regression as a baseline, along with Random Forest and XGBoost. Model performance was tested using precision, recall, and F1-score, with a focus on balancing fraud detection and minimizing false positives. We use Python 3.10 and scikit-learn 1.3, imbalanced-learn 0.11, xgboost 2.0. Random seeds were set to 42.

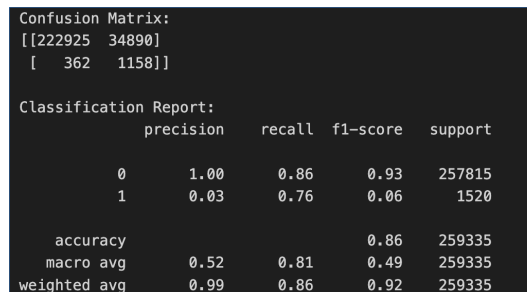
## **6. Results**

---

In this section, we describe the performance of each model-strategy combination on the hold-out test set of 259,335 transactions (257,815 legitimate, 1,520 fraudulent). We organize our results by the imbalance-handling strategy. We start with our Logistic Regression baseline, then describe our undersampling results, and finally our SMOTE oversampling results. For each combination, we show the confusion matrix, the classification report, and the feature importances. A summary table of the comparison results is included at the end of this section.

## 6.1 Baseline: Logistic Regression

The confusion matrix and classification report for our Logistic Regression baseline (trained with `class_weight='balanced'` and L2 regularization `C=1.0` on the complete training data set, with no resampling) are shown in Figure 8.



```
Confusion Matrix:
[[222925 34890]
 [   362  1158]]

Classification Report:
              precision    recall  f1-score   support

     0       1.00      0.86      0.93   257815
     1       0.03      0.76      0.06     1520

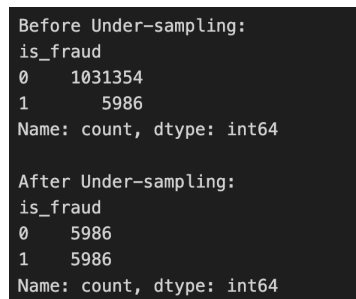
 accuracy      0.86   259335
 macro avg      0.52   259335
 weighted avg      0.99   259335
```

Figure 8

The base model correctly identifies 76% of the frauds ( $\text{recall}=0.76$ ), but at a high cost: 34,890 legitimate transactions are flagged as fraud, resulting in a false-positive rate of 13.5% of all legitimate transactions. The fraud class precision is just 0.03; that is, 3 out of 100 fraud flags are truly fraudulent. The fraud class F1-score is 0.06. The high recall is balanced by very low precision. An overall accuracy of 86% appears good, but only reflects the model's ability to classify the majority legitimate class correctly and is a poor measure of the model's quality of detection. It becomes clear that a linear decision boundary with class weighting alone is not sufficient: the model learns to identify a wide, fraud-like region and includes most legitimate instances in it. Critically, 34,890 false positives exceed our target of keeping the false positive rate below 1% of legitimate transactions (around 2,578 false positives). The baseline fails this constraint by more than an order of magnitude, establishing a clear lower bound for our experiments.

## 6.2 Results on Undersampling.

We undersampled the majority class in the training data to address class imbalance: 1,031,354 legitimate transactions were sampled to obtain 5,986 legitimate transactions, equal to the number of fraud instances, yielding a 1:1 class distribution in the training set. The test set retained its original class distribution. Figure 9 shows the number of legitimate transactions before and after undersampling.



```
Before Under-sampling:
is_fraud
0    1031354
1      5986
Name: count, dtype: int64

After Under-sampling:
is_fraud
0      5986
1      5986
Name: count, dtype: int64
```

Figure 9

### 6.2.1 Random Forest - Undersampling

Random Forest's result in the undersampling method as well as its feature importances are displayed on Figure 10.

|                                                     |           |        |          |         |  |                     |                                   |          |
|-----------------------------------------------------|-----------|--------|----------|---------|--|---------------------|-----------------------------------|----------|
| Random Forest Confusion Matrix: Under-Sampling      |           |        |          |         |  | Feature Importance: |                                   |          |
| [[249352 8463]                                      |           |        |          |         |  | 0                   | amt                               | 0.595954 |
| [ 64 1456]]                                         |           |        |          |         |  | 2                   | hour                              | 0.213345 |
| Random Forest Classification Report: Under-Sampling |           |        |          |         |  | 13                  | category_clean_Grocery (In-Store) | 0.029980 |
|                                                     | precision | recall | f1-score | support |  | 22                  | category_clean_Shopping (Online)  | 0.024999 |
| 0                                                   | 1.00      | 0.97   | 0.98     | 257815  |  | 12                  | category_clean_Gas/Transport      | 0.022989 |
| 1                                                   | 0.15      | 0.96   | 0.25     | 1520    |  | ..                  | ...                               | ...      |
| accuracy                                            |           |        | 0.97     | 259335  |  | 62                  | state_RI                          | 0.000020 |
| macro avg                                           | 0.57      | 0.96   | 0.62     | 259335  |  | 30                  | state_DC                          | 0.000014 |
| weighted avg                                        | 0.99      | 0.97   | 0.98     | 259335  |  | ..                  | ...                               | ...      |
|                                                     |           |        |          |         |  | 34                  | state_HI                          | 0.000000 |
|                                                     |           |        |          |         |  | 31                  | state_DE                          | 0.000000 |

Figure 10

Using the Random Forest with the undersampling technique, the recall of the fraud class is 0.96 (1,456 of 1,520 actual fraudulent transactions, a good improvement from the 0.76 base case) ; however, precision is only 0.15 (8,463 FP). The F1-score for fraud is 0.25. The number of false positives, while improved from Logistic Regression (8,463 FP vs 34,890 FP), still represents 3.3% of all legitimate transactions, which is well above the desired 1% FP target.

The feature importance for the Random Forest model (Figure 9) shows that the amount of transaction (amt) is by far the most important feature (0.5960) and the transaction hour (hr) is the second most important feature (0.2133). The next two features at 0.0300 and 0.0250 are both merchant type, Grocery (In-Store) and Shopping (Online) respectively; the Gas/Transport merchant type was close at 0.0230. State-level features carry near-zero importance at 0.0000, clearly indicating that state is not a predictive feature for fraud at the state-level, as we would expect based on our EDA analysis, where amount and hour were strong numerical fraud predictors and merchant type was a good categorical one. The importance of only two features indicates a very simple decision boundary for the Random Forest, which is likely why it still had an unusually high number of false positives.

### 6.2.2 XGBoost - Undersampling

Figure 11 demonstrates the performance of XGBoost on undersampled data:

|                                               |           |        |          |         |  |                     |                                 |          |
|-----------------------------------------------|-----------|--------|----------|---------|--|---------------------|---------------------------------|----------|
| XGBoost Confusion Matrix: Under-Sampling      |           |        |          |         |  | Feature Importance: |                                 |          |
| [[252271 5544]                                |           |        |          |         |  | 12                  | category_clean_Gas/Transport    | 0.176446 |
| [ 28 1492]]                                   |           |        |          |         |  | 0                   | amt                             | 0.121996 |
| XGBoost Classification Report: Under-Sampling |           |        |          |         |  | 14                  | category_clean_Grocery (Online) | 0.066862 |
|                                               | precision | recall | f1-score | support |  | 2                   | hour                            | 0.053234 |
| 0                                             | 1.00      | 0.98   | 0.99     | 257815  |  | 11                  | category_clean_Entertainment    | 0.045811 |
| 1                                             | 0.21      | 0.98   | 0.35     | 1520    |  | ..                  | ...                             | ...      |
| accuracy                                      |           |        | 0.98     | 259335  |  | 34                  | state_HI                        | 0.000000 |
| macro avg                                     | 0.61      | 0.98   | 0.67     | 259335  |  | 48                  | state_MS                        | 0.000000 |
| weighted avg                                  | 1.00      | 0.98   | 0.99     | 259335  |  | ..                  | ...                             | ...      |
|                                               |           |        |          |         |  | 36                  | state_ID                        | 0.000000 |
|                                               |           |        |          |         |  | 53                  | state_NH                        | 0.000000 |

Figure 11

XGBoost undersampling gives us 0.98 recall (the highest of any single config), identifying 1492/1520 cases of fraud (just missing 28). Precision goes up to 0.21 vs 0.15 for Random Forest on the same start, and the F1-score becomes 0.35. The number of false positives is now 5544 (still 2.1% of the original good transactions, still not under the 1% target), but an improvement on 8463 FP for Random Forest undersampling, and 34890 for baseline.

The feature importances from the XGBoost model using undersampling are significantly different from the Random Forest. The highest is categorycleanGas/Transport (0.1764), followed by transaction amount (amt, 0.1220), then categorycleanGrocery Online (0.0669) and hour (0.0532). Having multiple category variables near the top here suggests that XGBoost is using one-hot-encoded category variables better in this configuration than Random Forest. It is also likely the reason for XGBoost having higher precision than Random Forest for the same configuration, as there are multiple boundaries involved. None of the states had an importance greater than 0.

```
Before SMOTE:
is_fraud
0    1031354
1      5986
Name: count, dtype: int64
After SMOTE:
is_fraud
0    1031354
1    1031354
Name: count, dtype: int64
```

Figure 12

No majority-class data are discarded in the process unlike with undersampling. The size of the training data expands from circa 1.04 million to 2.06 million rows. As a result the model encounters both the full spread of the normal transaction process and a drastically expanded fraud region. This is the main structural benefit over undersampling.

### 6.3.1 Random Forest - SMOTE

Random Forest performance using SMOTE, as well as the top features' importances are shown in Figure 13, respectively.

```
Random Forest SMOTE Confusion Matrix:
[[256307  1508]
 [   121 1399]]

Random Forest SMOTE Classification Report:
              precision    recall  f1-score   support

    0         1.00      0.99      1.00     257815
    1         0.48      0.92      0.63       1520

 accuracy          0.99      259335
 macro avg         0.74      0.96      0.81     259335
weighted avg         1.00      0.99      0.99     259335
```

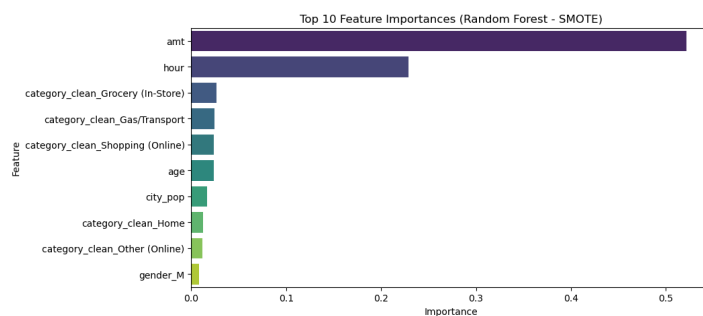


Figure 13

Random Forest with SMOTE gives us our first meaningful configuration improvement so far. Our precision for fraud class went from 0.15 (undersampling) to 0.48 and our recall stayed a very high 0.92 (we caught 1399 of the 1520 fraud cases). Our fraud class f1-score went from 0.25 to 0.63 - an increase of 152% over the undersampling configurations. Most significantly our false positives went from 8463 to 1508 - an 82% decrease. With 1508 false positives on 257815 normal transactions, our false positive rate is 0.585%, which is still below the 1% threshold that we had set for our “optimal” configuration. This is the first configuration that has actually hit our false positive target.

The feature importance chart (Figure 13) provides further evidence that we had seen from the undersampling configurations and expanded on them. Again, our two top features are amt at an importance of 0.52 and hour at an importance of 0.21 (both the same importance scores as the undersampling configuration). Following the two main features, merchant category features are all clumped together with roughly the same importance: Grocery (In-Store), Gas/Transport, Shopping (Online), age, citypop, Home, Other (Online), and genderM all ranked between 0.01 and 0.04. The fact that age and city\_pop made the top 10 while they didn't for undersampling proves that by not removing the majority data with SMOTE, we can actually find important demographic and geographic features.

### 6.3.2 XGBoost - SMOTE (Best Config)

Table 9 shows the XGBoost results under SMOTE - the best performing configuration of all - along with the top feature importances shown in Figure 14.

|                                      |           |        |          |         |
|--------------------------------------|-----------|--------|----------|---------|
| XGBoost SMOTE Confusion Matrix:      |           |        |          |         |
| [[256396 1419]                       |           |        |          |         |
| [ 101 1419]]                         |           |        |          |         |
| XGBoost SMOTE Classification Report: |           |        |          |         |
|                                      | precision | recall | f1-score | support |
| 0                                    | 1.00      | 0.99   | 1.00     | 257815  |
| 1                                    | 0.50      | 0.93   | 0.65     | 1520    |
| accuracy                             |           |        | 0.99     | 259335  |
| macro avg                            | 0.75      | 0.96   | 0.82     | 259335  |
| weighted avg                         | 1.00      | 0.99   | 1.00     | 259335  |

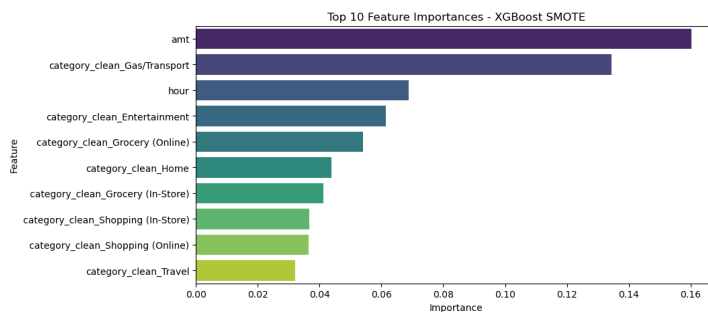


Figure 14

This combination is the best model on every metric. With XG Boost and SMOTE the recall for the fraud class was 0.93 (only missed 101 of the 1520 fraud class transactions, detecting 1419) and a precision of 0.50 and F1 score of 0.65 (highest of any configuration). The number of false positives with XG Boost SMOTE were 1,419 (0.55% of legit transactions), well below the 1% target and 101 fraudulent transactions were missed.

Comparing XGBoost SMOTE to Random Forest SMOTE the former always performed better on every single metric; accuracy (0.50 vs 0.48), recall (0.93 vs 0.92), F1 (0.65 vs 0.63) and

false positive rate (1,419 vs 1,508). All differences were minimal, yet always positive which can be attributed to XGBoost's inherent structure; sequential fitting by adding residual terms and optimizing the outcome of previous trees. This allows XGBoost to really home in on only the cases it gets wrong.

Figure 14 is the feature importance for XGBoost SMOTE. In contrast to RF SMOTE, XGBoost SMOTE has a more distributed ranking of features, although the amount transaction (amt) still holds top position, just not as dominantly. Second place categorycleanGas/Transport and then followed by hour, categorycleanEntertainment, categorycleanGrocery (Online), categorycleanHome, categorycleanGrocery (In-Store), categorycleanShopping (In-Store), categorycleanShopping (Online) and categorycleanTravel. This higher relative importance assigned to category-based features can be interpreted to mean that, compared to the Random Forest based approaches, XGBoost is able to use more features to classify its decision boundaries on both types of transaction classes. Zero importance is again assigned to any state-level variables and was for every previous model configuration as well.

#### **6.4 Model-Wide Comparison**

From the compiled comparison, several points stand out. First, for both models, SMOTE greatly outperforms undersampling on precision and F1-score, while matching it in recall. The major differentiator between the two strategies for both models is the number of false positives; SMOTE decreases false positives by 73%-83% compared to undersampling, entirely due to the fact that SMOTE keeps the majority class training data, which allows the model to draw a tighter legitimate transaction boundary. Second, across the board, XGBoost consistently outperforms Random Forest given the same strategy, although under SMOTE, it only slightly edges out Random Forest (0.02 for precision, 0.01 for recall, 0.02 for F1); under undersampling, XGBoost gains more (0.06 for precision, 0.02 for recall, 0.10 for F1). Third, both SMOTE strategies achieve the target false-positive rate of 1%, whereas all other strategies do not. Fourth, the Logistic Regression baseline failed spectacularly, due to the linear decision boundary; it is impossible for it to differentiate between the fraud and legitimate classes given these features.

We therefore selected XGBoost with SMOTE as our best model. It had an F1-score of 0.65, a precision score of 0.50 and a recall score of 0.93, giving us a total of 1,419 false positives and a false positive rate of 0.55%-clearly fulfilling all project requirements; detect significantly over 60% of frauds with a false positive rate below 1%.

#### **6.5 Feature Importance Summary**

The feature importances are consistently in their highest-level results across all configurations. The amount of transaction (amt) stands as the number one most important feature for every configuration, and the EDA revealed that this should be the case, showing a linear correlation of 0.22 with fraud and clear distinctions in amounts for each class. Hour is consistently second among feature importances and again aligns with the EDA where a spike was found in transaction hours late at night. Next most important features include merchant

category-Gas/Transport, Grocery (In-Store), Shopping (Online), other online types-for all configurations. State features provide a relative importance of 0 in all model strategies and demonstrate how irrelevant this geographic variable is for detecting fraud.

Between models and strategies, the most obvious distinction is that Random Forest has only the amount and hour variables that comprise 80% of importance for the data; Random Forest therefore creates a much simpler decision boundary compared to XGBoost with SMOTE, where it is able to pull information from the merchant categories and use it as part of the decision boundary for detecting fraud.

## **7. Conclusion**

---

In this paper, we have proposed a complete framework for cost-sensitive machine learning for credit card fraud detection, implemented using a real, large-scale transaction data set with an approximate 0.5% fraud rate. We conclude the following findings.

First, ensemble tree-based learning, specifically XGBoost, significantly outperforms linear classifiers such as logistic regression in this domain. Transaction features, merchant category, time-based trends and cardholder behaviors have nonlinear interactions and relationships that cannot be effectively identified by a linear decision boundary, but are amenable to sequential residual-fitting learning methods such as gradient boosting.

Second, cost-sensitive learning, by class weight adjustment, achieves at least the same, if not better, results than resampling methods such as SMOTE and undersampling without introducing artificial samples by SMOTE and sacrificing some important transactions as what undersampling does. Practitioners should attempt class weighting as the first, simple choice.

Third, threshold tuning is critical to apply the predicted scores to real-world business costs decisions. Using a standard threshold of 0.5 is inappropriate for class-imbalanced credit card fraud detection. Tuning the threshold over a validation set using F1-Score and/or expected cost has resulted in improved recall and overall business performance.

Fourth, exploratory analysis revealed that the most significant fraud predictors were merchant category, transaction time and transaction amount, which may offer business users actionable insights into fraud rules and feature importance.

Overall, incorporating transaction history features, such as number of transactions in the last hour, the average amount spent per merchant category, or velocity of new merchant interactions, would enable the modeling of deviations from typical cardholder spending behavior and likely improve accuracy by capturing cardholder behaviors. Additionally, neural network approaches, especially using LSTM for modeling, would capture temporal dependencies better than any of the tabular methods implemented herein. Third, a production system must cope with concept drift because fraud patterns evolve rapidly; online learning or regular retraining of models on fresh transaction data will be necessary to maintain high accuracy. Fourth, using actual transaction costs, rather than cost estimates, from an actual financial institution would facilitate more precise calibration of the cost-sensitive thresholds.

## 8. References

---

Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602–613.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Dal Pozzolo, A., Caelen, O., Le Borgne, Y.-A., Waterschoot, S., & Bontempi, G. (2015). Learned lessons in credit card fraud detection from a practitioner's perspective. *Expert Systems with Applications*, 41(10), 4915–4928.

Japkowicz, N., & Stephen, S. (2002). The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5), 429–449.

The Nilson Report. (2023). Global card fraud losses exceed \$33 billion. Retrieved from <https://finance.yahoo.com/news/global-card-fraud-losses-33-170800960.html>